02207 PyUVM Compendium

Version 1.0.3.0

SyoSil ApS

Oct 06, 2026

CONTENTS:

1 Instance Hierarchy **1**
- 1.1 Components, parents, and children . . . 1
- 1.2 Instance paths . . . 2
- 1.3 Wildcard path patterns . . . 3

2 ConfigDB **4**
- 2.1 Context and instance paths . . . 4
- 2.2 ConfigDB set() . . . 5
- 2.3 ConfigDB get() . . . 5
- 2.4 Why use the ConfigDB instead of direct assignments? . . . 5

CHAPTER

ONE

INSTANCE HIERARCHY

The `uvm_components` of a pyuvm testbench form an instance hierarchy. It describes the parent-child relationships between all `uvm_components` in a testbench.

Objects derived from `uvm_object` are not part of the instance hierarchy.

1.1 Components, parents, and children

When a `uvm_component` is created using the factory with the `create()` class method, the `name` and `parent` of the component are set.

```
class default_test(uvm_test):
   ...
   def build_phase(self):
      ...
      self.env = env.create(
          name   = "env",
          parent = self,
      )
```

This creates a environment component with the name `env` as a child of the test. The instance of the test in a pyuvm testbench is created by `uvm_root` and is always named `uvm_test_top`.

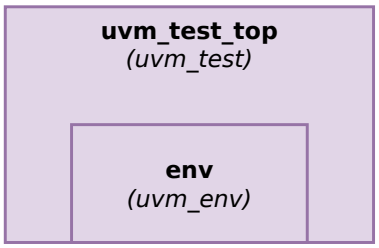


Fig. 1.1: The `env` component is a child of `uvm_test_top`.

1.2 Instance paths

An instance path is the dot-separated sequence of instance names from the top component (`uvm_test_top`) to a particular component.

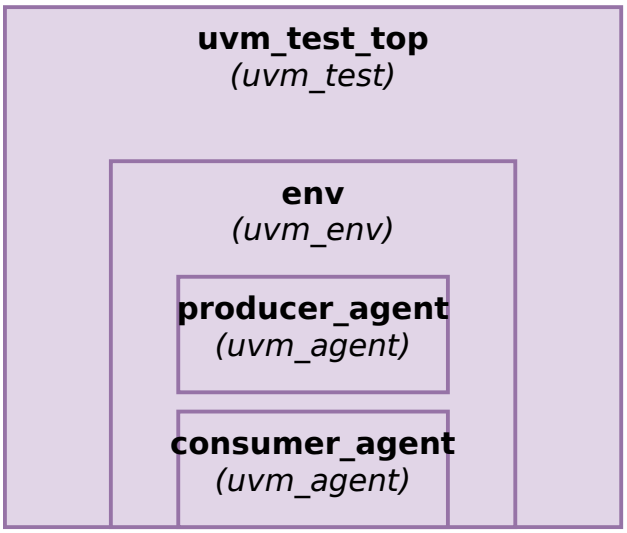


Fig. 1.2: Deeper nested instance hierarchy with two `uvm_agents`.

The example from above creates this hierarchy:

```
uvm_root
└── uvm_test_top
    └──  env
       ├── producer_agent
       └── consumer_agent
```

The corresponding full instance paths are:

Component	Full instance path
test	`uvm_test_top`
environment	`uvm_test_top.env`
input agent	`uvm_test_top.env.input_agent`
input monitor	`uvm_test_top.env.input_agent.monitor`
output monitor	`uvm_test_top.env.output_agent.monitor`

To get the name or full instance path of a component the following methods are available.

- `get_name()` returns only the local instance name.
- `get_full_name()` returns the full instance path.

1.3 Wildcard path patterns

Path-based methods such as `ConfigDB`'s `set()` and `uvm_factory`'s `override_instance_by_<type/name>` can match a concrete instance path against a wildcard pattern[1]. This addresses a region of the hierarchy without requiring a reference to every component in that region. Most importantly this allows the use of `*` which matches against everything. This allows a instance override for example to target all components within the environment:

```
# In the env: replace all SequenceItem classes with NewSequenceItem
uvm_factory().set_inst_override_by_type(
    original_type_name = SequenceItem,
    override_type_name = NewSequenceItem,
    full_inst_path     = "uvm_test_top.env.*",
)
```

Similarly the test can store a value in the `ConfigDB` for all descendants of `env`:

```
ConfigDB().set(
   context    = self,
   inst_name  = "env.*",
   field_name = "timeout_cycles",
   value      = 100,
)
```

[1] Pyuvm specifically uses fnmatch for matching wildcard patterns.

CHAPTER

TWO

CONFIGDB

The configuration database, `ConfigDB`, is a shared, path-based database. Components can store a value using `set()`, and retrieve it using `get()`. The value is identified by a field name, similar to a dictionary, and an instance path. A key value pair is only 'visible' to a single (or multiple) components.

In pyuvm the `ConfigDB` is a singleton object. `ConfigDB()` always accesses the same database object.

2.1 Context and instance paths

Both `set()` and `get()` take two arguments to resolve the effective lookup path.

1. `context`, an object of type `uvm_component`
2. `inst_name`, a string

The effective lookup path is resolved from the full instance path of `context` with the `inst_name` appended. So for a `ConfigDB` call with

- `context`: `sat_default_test` (full instance path: `"uvm_test_top"`)
- `inst_name`: `"env"`

Produces `"uvm_test_top.env"` as the effective lookup path.

The `context` should almost always be the current component referenced by `self`. Then, from the perspective of the `context`, `inst_name` selects the child or descendant to configure. From the receiving component's perspective (again passing itself as `context`) using `inst_name=""` selects its own full instance path. For example, if `env` is a child of `uvm_test_top`:

```
# In the test (uvm_test_top):
ConfigDB().set(
    context    = self,
    inst_path  = "env",
    field_name = "cfg",
    value      = self.cfg,
)

# In the environment (uvm_test_top.env):
self.cfg = ConfigDB().get(
    context    = self,
    inst_path  =  "",
    field_name ="cfg",
)
```

Both calls resolve to `uvm_test_top.env`, so the environment retrieves the config. Using itself as context means that the environment does not need to specify its full path or know the name of the test above it.

`set()` also accepts wildcards in the `inst_name` see *Wildcard path patterns*.

2.2 ConfigDB set()

```
def set(context, inst_name, field_name, value)
```

Stores a value under `field_name` at the path or path pattern formed from `context` and `inst_name`. The `inst_name` is relative to the context component's full instance path. With `context=None`, it is a full path or path pattern from the root.

Arguments:

- `context`: A `uvm_component` used as the base for the path, or `None` to use the root context.
- `inst_name`: An instance path relative to `context`. Use `""` for the context component itself. Wildcards are allowed.
- `field_name`: The key at which to store the value.
- `value`: The value to store.
- Returns: `None`.

2.3 ConfigDB get()

```
def get(context, inst_name, field_name, default=default_get)
```

Retrieves a value by matching the concrete path formed from `context` and `inst_name` against the stored paths and path patterns, then looking up `field_name`. The `inst_name` is relative to the context component's full instance path. With `context=None`, it is a full path from the root.

Arguments:

- `context`: A `uvm_component` used as the base for the path, or `None` to use the root context.
- `inst_name`: A concrete instance path relative to `context`, without wildcards.
- `field_name`: The key for the value to retrieve.
- `default`: The value to return if no matching configuration value is found.
- Returns: The stored value, or the supplied `default` if no matching configuration value is found.

2.4 Why use the ConfigDB instead of direct assignments?

A direct assignment such as `self.env.driver.timeout_cycles = 100` requires the test to have access to the driver object and to know which attribute to change. The driver must also exist before the assignment can be made. This couples the test to the Python object structure of the environment.

With the `ConfigDB`, the test specifies a value and the instance path where it should be available. The receiving component retrieves the value under an agreed field name and assigns its own attributes. The value can be stored before the receiving component exists, because the database stores paths rather than looking up component objects when `set()`

is called. This is useful during `build_phase()`, when parents configure components that will be created further down the hierarchy.

The receiving component does not need to know the name of the test or how to navigate the Python references above it. It can retrieve configuration relative to its own instance path, making it easier to reuse in another environment. The test and component must still agree on the field name, the meaning of the value, and a matching instance path.